ИТМО

AutoText2SQL

Система автоматизированного создания Text2SQL-инструментов

Владислав Кинякин
Никита Студеникин

Проблема

Универсальные Text2SQL не знают вашу БД

Universal Text2SQL

- Обучены на BIRD / Spider / WikiSQL
- Не знают конкретную пользовательскую БД
- Галлюцинируют имена колонок
- Неправильные JOIN-связи
- Игнорируют domain-специфику (форматы дат, бизнес-правила)

Альтернатива: FT под каждую БД

- Дни-недели ручной разметки
- Требуются экспертные SQL-разработчики
- Под каждую новую БД — заново
- Не масштабируется на 10+ БД

Нужен способ автоматически адаптировать Text2SQL под произвольную БД

Цели и задачи

Цель практики исследовать возможность построения Text-to-SQL инструмента, превосходящий базовый (~25% BIRD / ~50% Ambrosia), решая обе задачи одновременно — генерацию корректного SQL и корректную реакцию на неоднозначные запросы.

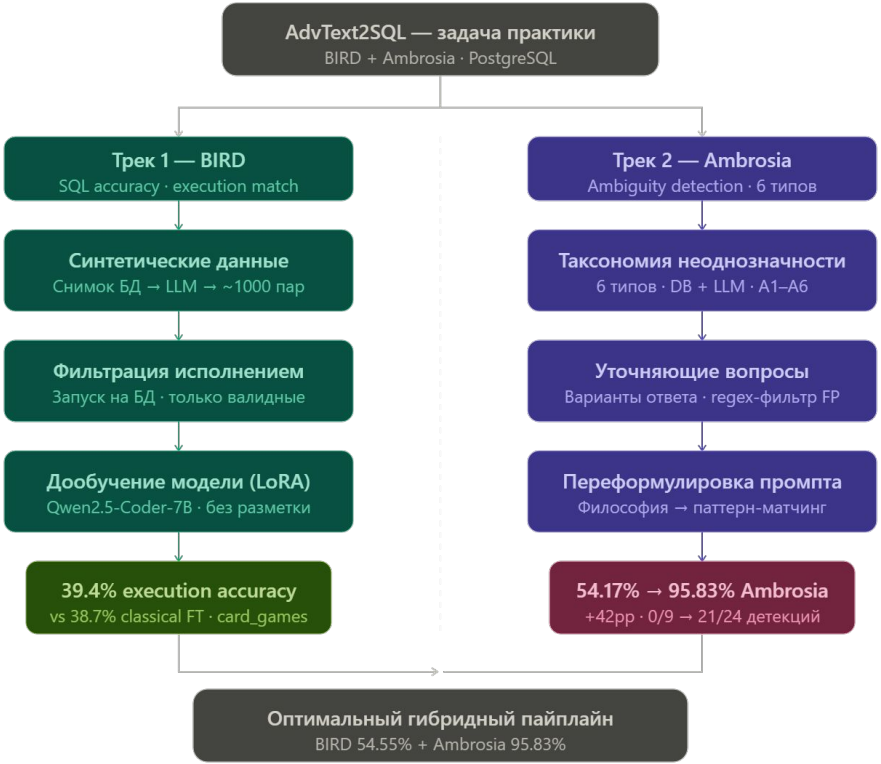
Задачи:

- исследовать материалы по существующим решениям.
- провести эксперименты на бенчмарке BIRD
- провести эксперименты на бенчмарке Ambrosia
- найти оптимальные конфигурации для каждого бенчмарка
- объединить лучшие решения
- провести feature engineering на всех имеющихся фичах
- провести исследования эффективности finetune для решения задачи text2sql на случайных БД

Будущее проекта:

- MCP оболочка с объединенными методиками
- тестирования на реальных БД

Отправная точка



Feature Engineering на 34 фичах

ИТМО

Группа А: Обогащение схемы

Статистика колонок min/max/top-K	+9.09% BIRD
TSV вместо JSON — экономия 30% токенов	+4.55% BIRD
FK/PK в промпте — перегрузка JOIN-ами	−9.09% BIRD

Группа Б: Качество SQL-генерации

CoT декомпозиция на под-вопросы	+4.55% BIRD
Классификация сложности + адаптивный промпт	+4.55% BIRD
Fallback при сбое детектора неоднозначности	+4.55% BIRD

B: AmbiSQL — детекция неоднозначности

Фича	BIRD Δ	AMB Δ
Taxonomy LLM (классификация A1–A6)	0%	−8.34%*
Уточняющий вопрос с вариантами	−4.55%	−8.34%*

**В isolation — ошибки фреймирования промпта. В комбо → AMB 91–96%*

Прогрессия и финальные числа

ИТМО

Финальные числа (оптимум: 14 фичей)

Датасет	Baseline	Результат	Δ абс.	Δ отн.	Примечание
BIRD small (22 вопр.)	36.36%	**54.55%**	+18.19pp	+50%	14 фичей · потолок ~59%
BIRD large (241 вопр.)	~36%	41.49%	+5pp	—	simple 57% · moderate 39% · challenging 21%
Ambrosia small (24 вопр.)	54.17%	**95.83%**	+41.66pp	+77%	F1 = 88.2% по типам
Ambrosia large (240 вопр.)	54.17%	91.24%	+37.07pp	+68%	detect-only режим (без auto-resolve)

Динамический pipeline

ИТМО

Проблема: один статический набор из 14 фичей применяется ко всем запросам — от COUNT(*) до 4-table JOIN. Фичи не-аддитивны: комбо даёт +12pp вместо суммы +18pp.

Phase 1 Rule-based selector

- Keyword heuristics
- Без ML — быстро
- Базовая линия для Phase 2

Phase 2 Multilabel classifier

- sentence-transformer → Linear(34) + Sigmoid
- BCEWithLogitsLoss
- 241 × 34 attribution матрица

Phase 3 DB-aware scoring

- query_score × db_score
- Учёт характеристик конкретной БД

Гипотеза: dynamic pipeline > best-static-combo > GPT-5.2-mini baseline

Решение

Автономный пайплайн: 5 стадий, ни одного ручного шага

1 **PROFILE** *read DB metadata*

2 **GENERATE** *LLM + templates*

3 **VALIDATE** *exec filter*

4 **FINE-TUNE** *LoRA / Qwen-7B*

5 **SERVE** *vLLM / MCP*

Вход

`postgresql://...`

Выход

`LoRA-адаптер + MCP-сервис`

~30 минут от URL базы до работающего сервиса. Идемпотентно, артефакты на диске, независимый дебаг.

Архитектура

6 модулей, ~5000 строк, 35 unit-тестов

profiler/	DBConnector, SchemaExtractor, StatsCollector, SampleCollector — вычитка метаданных PG
synth/	LLMSyntheticGenerator + TemplateSyntheticGenerator (25 шаблонов) — генерация Q-SQL
synth/Validator	Parse (sqlglot) + Whitelist (SELECT) + Execute на реальной БД + опц. LLM-judge
training/	DatasetBuilder (chat-format JSONL) + notebook_generator (Colab template)
serve/	Dockerfile.vllm + LoRA hot-swap + Text2SQLClient SDK + QueryLogger
heal/	HealCollector (failed.jsonl → дедуп) + HealSQLGenerator (GPT-4 корректор)

Артефакты передаются через файлы на диске → идемпотентность, возобновляемость, независимый дебаг каждой стадии

Стадия 1: Profiler

Что мы знаем про БД

Собираемые метаданные

- Структура: таблицы, колонки, типы, FK
- Статистика numeric: min / max / avg / median
- Статистика categorical: top-5 значений + частоты
- Date: min / max диапазон
- Sample-rows: 3 реальные строки / таблица
- Low-cardinality (enum) — полный список значений

Пример profile.json

```
"schema_str": "TABLE cards\n  id BIGINT PK\n  artist
TEXT ..."
"relationships_str": "cards.uuid →
foreign_data.uuid"
"column_stats_str":
   "cards.artist: 252 distinct,
    top=[John Avon:1128, Kev Walker:997]"
"sample_rows": {"cards": [{...}, {...}, {...}]}
"low_cardinality_values":
   {"cards.bordercolor":
    ["black","white","gold","silver"]}
"complexity_score": 27.82
```

card_games (115 колонок) → profile.json за ~30 секунд / ~120 SELECT-ов к системным таблицам

Стадия 2: Генерация

Два пути — стабильность + разнообразие

Template generator

- 25 SQL-шаблонов разной сложности
- 100% валидный SQL — нет галлюцинаций
- Бесплатно — без API-вызовов
- Категории: lookup, count, top-N, JOIN, GROUP BY, CTE, window
- Подставляет реальные имена и значения из profile.json

LLM generator

- GPT-4 / 4o / 4.1 / codex-mini как teacher-модели
- Естественные natural-language вопросы
- Сложные SQL: CTE, окна, подзапросы
- Параметр language: ru / en (train==inference invariant)
- Стоит денег, требует execution-валидации

Используем оба источника параллельно — Template даёт фундамент, LLM — разнообразие

Стадия 3: Валидация

Защита от ошибочных примеров в три уровня

ИТМО

1 **Parse**

sqlglot разбирает SQL как PostgreSQL. Отсев синтаксических ошибок.

2 **Whitelist**

Только SELECT. Запрет DROP / DELETE / UPDATE / TRUNCATE.

3 **Execute**

Выполнение на реальной БД. Должен вернуть ≥1 строку, без ошибок.

4 **LLM Judge**

Опц. GPT-4 проверяет семантическое соответствие question↔SQL.

Пример реального фильтра: **510 raw → 468 passed (93%)** reasons: sql_error=23, too_few_rows=11

Стадия 4: Fine-tuning

LoRA-адаптеры на Qwen2.5-Coder-7B

Параметр	Значение
База	Qwen2.5-Coder-7B-Instruct (4-bit)
Метод	LoRA
r / α	16 / 32
Learning rate	2e-4
Epochs	1
Batch × grad_accum	1 × 8
max_seq_length	12288
Target modules	q/k/v/o/gate/up/down_proj
GPU	A100 40GB (Colab)
Acceleration	unsloth ~2×

Ключевой инвариант

train == inference

- Системный промпт со схемой БД ≈ 10 000 токенов
- На контексте 4096 ответ обрезался — модель не училась SQL
- Подняли до 12288 — обучение работает корректно
- Eval отключён в trainer — OOM-фикс (val активации без чекпойнтинга)

~40 минут на одну модель × 5 генераторов = ~3.5 часа на полный эксперимент

Стадия 5: Production

От адаптера к работающему сервису

vLLM в Docker

LoRA hot-swap без merge; экономия диска; быстрая загрузка адаптеров

MCP-сервер (FastMCP)

Интеграция с AI-агентами (Claude, GPT) через Model Control Protocol

Python SDK

Text2SQLClient(api_url).query(question) → {sql, results}

QueryLogger middleware

queries.jsonl + failed.jsonl — основа для self-healing цикла

Hot-start через profile.json

<50 ms вместо 30+ секунд (нет повторного чтения схемы)

Self-healing loop (heal/)

failed.jsonl → GPT-4 корректор → corrections.jsonl → retraining

Эксперимент

Какая модель-генератор синтетики работает лучше всего?

ИТМО

Setup

Параметр	Значение
БД	card_games (6 таблиц, 115 колонок)
Test	BIRD card_games subset
Test-size	31 реальный вопрос
Метрика	Execution accuracy
Сравнение	Result sets (set equality)
Base FT	Qwen-Coder-7B (4-bit)
Train size	≈ 500–1000 пар / эксп.

5 генераторов синтетики

`gpt-4.1`	*топ*
`gpt-4o-mini`	*средний*
`gpt-5.1-codex-mini`	*code-spec*
`gpt-4.1-mini`	*средний*
`gpt-4.1-nano`	*бюджетный*

Главный вопрос: нужна ли сильная и дорогая LLM для генерации синтетики?

Результаты

Три итерации: RU → RU+EN → EN

Генератор	Эксп. 1 (RU)	Эксп. 2 (RU+EN)	Эксп. 3 (EN)
baseline (без FT)	19.4%	19.4%	19.4%
gpt-4.1	22.6%	32.4%	**39.4%**
gpt-4o-mini	12.9%	22.6%	**25.8%**
gpt-5.1-codex-mini	19.4%	22.6%	**25.8%**
gpt-4.1-mini	6.5%	19.4%	**22.6%**
gpt-4.1-nano	16.1%	16.1%	**18.5%**

Точка отсчёта (classical FT на полном BIRD train): **38.7%** → Эксп. 2 / gpt-4.1 = **39.4%** (обходит при 10× меньше данных, без человеческой разметки)

Ключевые выводы

5 наблюдений из трёх итераций

ИТМО

1

Влияние языка синтетики и языка валидации.

Эксп. 2 / gpt-4.1 (32.4%) и. Эксп 3 / gpt-4.1 (39.4%) . Вопросы на английском могут более четко пониматься при обучении.

2

Языковой инвариант train↔test критичен

Эксп. 1 → 2: добавление EN-синтетики дало +10 п.п. для gpt-4.1 (с 22.6% до 32.4%)

3

Сила исходной модели

baseline без FT: 19.4%

4

Эксп. 1 — методологическая находка, не failure

Зафиксировали языковой рассинхрон как явление и устранили в Эксп. 2

5

Комбинация языков

Комбинация языков в синтетике не является универсальной выборкой, нужно выбирать правильный язык при генерации синтетики

Сравнение с classical full-BIRD FT

Главный научный contribution

Параметр	Classical FT (Эксп. 0)	AutoText2SQL (Эксп. 2)
Объём данных	≈ 10 000 человеческих пар	**1000 автогенерированных пар**
Разметка	Ручная (дни — недели)	**Полностью автоматическая**
Применимость	Только под BIRD-БД	**Любая PostgreSQL-БД**
Время с нуля под новую БД	Недели разметки + обучение	**≈ 30 минут end-to-end**
Accuracy на BIRD card_games	**38.7%**	**39.4%**

Для одной БД автоматически сгенерированной синтетики достаточно, чтобы воспроизвести качество classical FT по всему BIRD, **при этом снять зависимость от человеческой разметки**

Что сделано · Что дальше

ИТМО

Сделано

- Полный автономный пайплайн (CLI, 7 команд)
- 3 итерации экспериментов
- Self-healing loop через failed-queries
- 35 unit-тестов + документация
- Сопоставимость с classical full-BIRD FT

Дальше

- vLLM-сервериング + MCP-обёртка
- Прогон на enterprise-БД (30+ таблиц)
- DPO / RLAIF поверх SFT-адаптера
- Адаптивный target_count под complexity_score
- Redact PII в low_cardinality перед отправкой в LLM
- Интеграция с реальной BI-системой
- Расширение на MySQL / Snowflake / BigQuery

Q & A

Спасибо за внимание!

ITMO more than a UNIVERSITY

Ваши контакты